

Mizar Evo

Readable, AI-Ready, Scalable Formal Mathematics

Eight problems, eight proposals

A discussion with the Bialystok Mizar team, September 2026

Mizar Evo project

September 2026

0.1 - A First Look

Legacy Mizar **exact MML excerpt**

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Mizar Evo **specification example**

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed view
  end;
end;
```

0.2 - The Proposal In One Sentence

Key Phrase:

*Preserve Mizar's mathematical vernacular.
Update the compiler, verifier, artifact, and publication layers.*

What this talk does not claim:

- Full MML migration is not complete.
- The language standard is not final.
- AI assistance is not a substitute for proof checking.
- We build on what the current Mizar system has achieved.

Every code example is labeled:

Label	Meaning
exact MML excerpt	exact current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	an example; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases.

0.4 - What We Need From This Visit

Key Points:

- find compatibility constraints that are hard to see outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should link to a package library;
- collect the objections we have not thought of.

Main question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

1.1 - What Mizar Got Right

Key Points:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;
- a well-established, carefully maintained library (MML 5.94.1493: 1493 articles);
- a tradition of publishing formal articles (Formalized Mathematics).

Message:

- We build on these strengths. We judge every proposal by whether it protects them.

1.2 - Pressure One: Scale

Key Points:

- MML has grown to roughly 1500 articles that depend on each other.
- The unit of dependency, review, and reuse is the whole article.
- Tools resolve article environments. But reviewers cannot see the resolved dependencies in the source.
- Whole-library maintenance operations (renames, refactorings, revisions) become more risky as the library grows.

Message:

- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

1.3 - Pressure Two: Tooling Expectations

Key Points:

- Editors are expected to give immediate feedback, even on incomplete or broken source.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work through packages with versions.
- Documentation is expected to be generated, linked, and browsable.

Message:

- Common programming tools already provide these features. New users expect formal libraries to provide them too.

1.4 - Pressure Three: AI

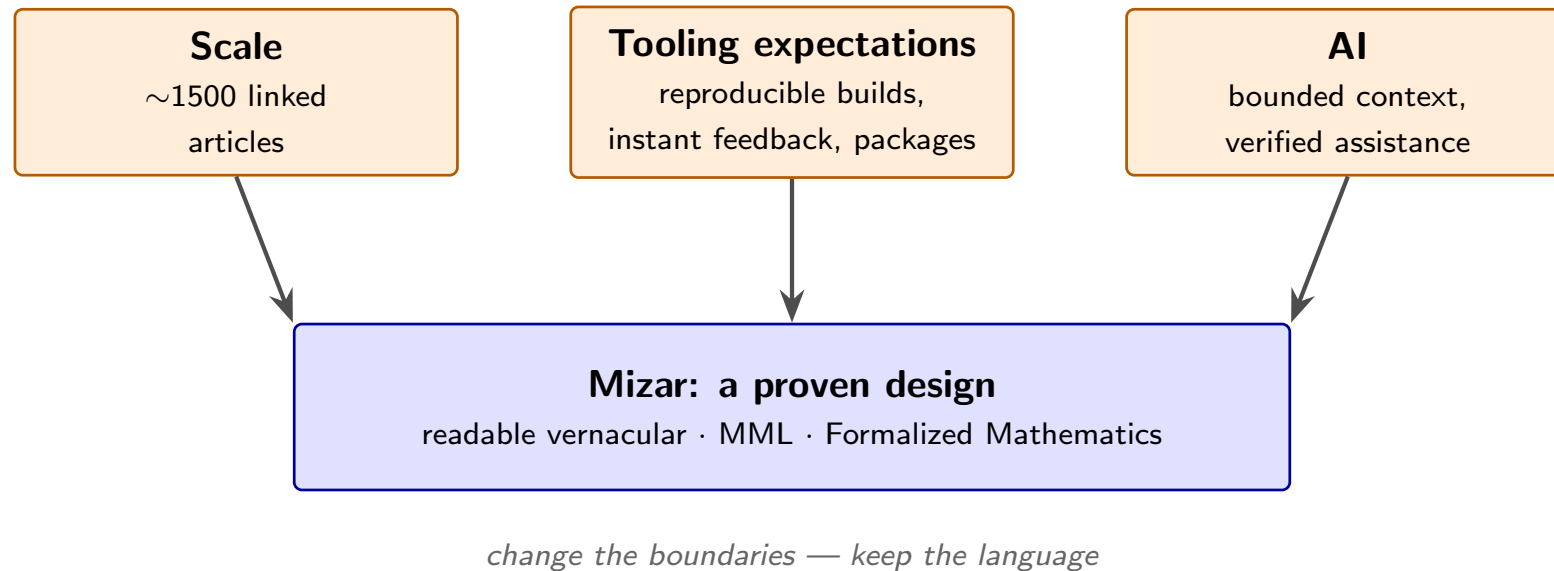
Key Points:

- AI agents are already useful for search, explanation, and repair.
- They need a limited amount of structured context linked to the source, rather than a copy of the whole library.
- Their output must never decide whether a proof is valid. Verification must stay independent of how capable the assistant is.
- Readable source helps here. AI editing and retrieval work best with stable local text patterns.

Message:

- Mizar's readability makes safe AI assistance possible today.

1.5 - Three Pressures, One Design



Message:

- These three needs do not mean that Mizar's design was wrong. Together, they give us reasons to change its boundaries.

1.6 - The Design Rule

Key Phrase:

*Do not make proofs harder to read to add automation.
Use automation to protect and extend readability.*

Three goals, one test:

Goal	Test for every feature
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see a limited context that we can audit?
Scalability	do boundaries stay stable as the library grows?

2.1 - The Problem

Legacy Mizar **exact MML excerpt**

```
environ
```

```
vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,  
             FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;  
notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,  
          CARD_1, STRUCT_0;  
constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;  
registrations ZFMISC_1, CARD_1, STRUCT_0;  
theorems STRUCT_0;
```

```
begin :: Additive structures
```

Key Points:

- Information about a symbol's origin is spread across several role lists. A reviewer cannot see which article provides which notation, constructor, or cluster.
- The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.
- Tools cannot cache or invalidate units smaller than an article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:

- Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

2.3 - The Evo Answer: Import Prelude

Mizar Evo **specification example**

```
import .function;  
import mml.algebra.structure.sorted;  
  
definition  
  let S be 1-sorted;  
  mode BinOpDef: BinOp of S is  
    Function of [: S.carrier, S.carrier :], S.carrier;  
end;
```

Rules that make this deterministic:

- all imports appear before the first item; no mid-file environment changes;
- imports provide the initial active lexicon. Every symbol, notation, registration, and theorem is traceable to exactly one import;
- stable fully-qualified names are derived from package and module paths.

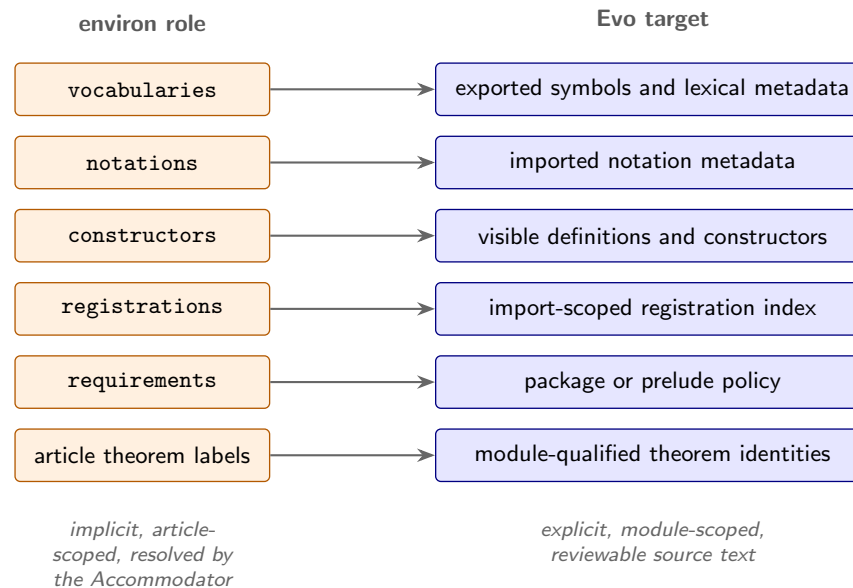
Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

Key Points:

- a manifest plus a lockfile makes every build reproducible;
- versioned reuse (SemVer) replaces manual copying between article sets.



Message:

- migration needs more than renaming. Current environments mix roles for semantics, syntax, and automation. Migration reports must explain what each imported module provides.

2.6 - What Is Preserved, What We Ask

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Bialystok:

- Which `environ` roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

3.1 - The Problem

Legacy Mizar **exact MML excerpt**

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Key Points:

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) depend on naming conventions;
- stored data and canonical values (a zero, a unit) are not distinguished.

Key Points:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- The syntax does not tell a migration tool whether a selector is intrinsic data, a canonical value with obligations, or an inherited view.
- Errors appear far from their cause, as type mismatches in later articles.

Message:

- At MML scale, structure inheritance is a graph maintenance problem, and the graph needs explicit edges that the verifier can check.

3.3 - The Evo Answer: Field, Property, Attribute

Mizar Evo **specification example**

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

Three distinct concepts:

Concept	Meaning	Consequence
field	intrinsic data supplied by the value	constructor shape, equality
property	uniquely determined canonical value	existence/uniqueness obligations
attribute	predicate-style refinement	cluster propagation, not layout

3.4 - The Evo Answer: Explicit Inheritance

Mizar Evo **specification example**

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed
  end;
end;
```

Key Points:

- one inherit statement per parent; mapping and renaming are source text;
- narrowing and type conversions carry coherence proof obligations;
- the additive/multiplicative naming convention becomes a checked view.

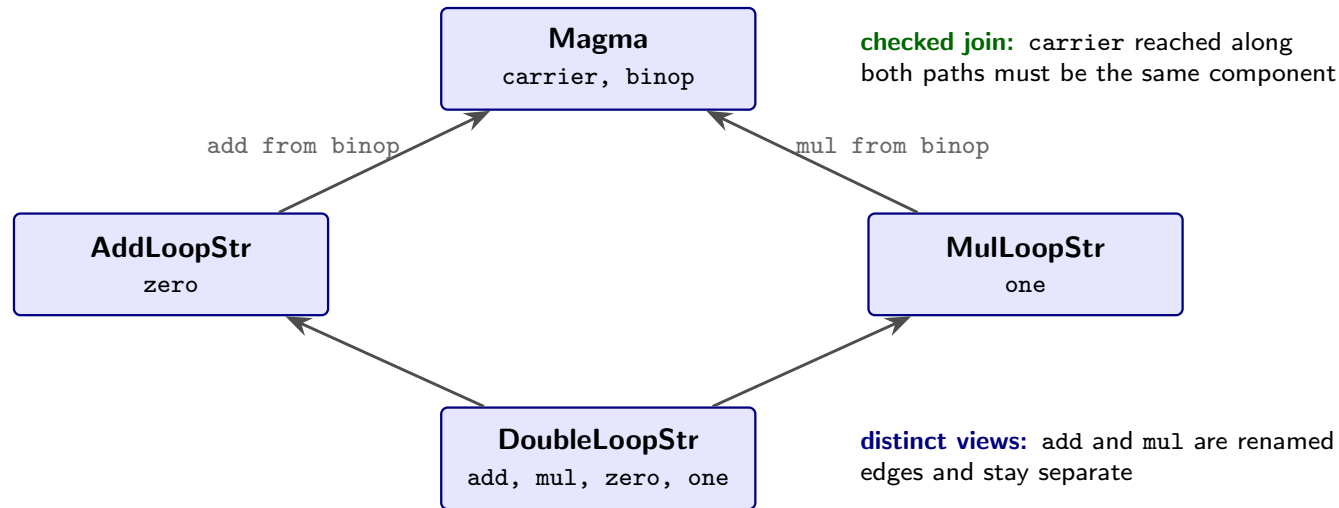
3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

Mizar Evo **specification example**

```
struct DoubleLoopStr where
  field carrier -> set;
  field add -> BinOp of carrier;
  field mul -> BinOp of carrier;
  property zero -> Element of carrier;
  property one -> Element of carrier;
end;

inherit DoubleLoopStr extends AddLoopStr;
inherit DoubleLoopStr extends MulLoopStr;
```

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)



the verifier traces from chains to root declarations — disagreement is a diagnostic with source spans

Message:

- the verifier traces from chains to root declarations; both paths must introduce the same components (the shared carrier above);
- diamond inheritance becomes a diagnostic with source spans, not a silent merge decided by declaration order.

3.6 - What Is Preserved, What We Ask

Preserved:

- structures remain Mizar structures: carriers, selectors, Element of;
- aggregates become longer only where we need to make a hidden decision explicit.

Questions for Bialystok:

- Is the field / property / attribute split readable in real algebraic articles, or does it require too many annotations in simple cases?
- Is one-parent-per-inherit acceptable for parts of MML with much inheritance, such as the ALGSTR and topology hierarchies?
- Which MML structures would be the best diamond test cases?

4.1 - The Problem

Legacy Mizar **exact MML excerpt**

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
Element
  of M;
  coherence;
end;
```

Key Points:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
- Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

- Automation without explanations makes a large library harder to maintain, even when it is sound.

4.3 - The Evo Answer: Labeled, Traceable Registrations

Mizar Evo **specification example**

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier stores an import-filtered cluster resolution graph;
- @show_resolution and explanation artifacts answer "why (not)?" with the actual chain, e.g. empty -> finite -> countable.

Mizar Evo **specification example**

```
registration
  let n be Nat;
  reduce NatAddZero: n + 0 to n;
  reducibility
  proof
    let n be Nat;
    thus n + 0 = n by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable reduce items.

4.5 - What Is Preserved, What We Ask

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- no new proof text is required at use sites - only traces are added.

Questions for Bialystok:

- Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?
- Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

5.1 - The Problem

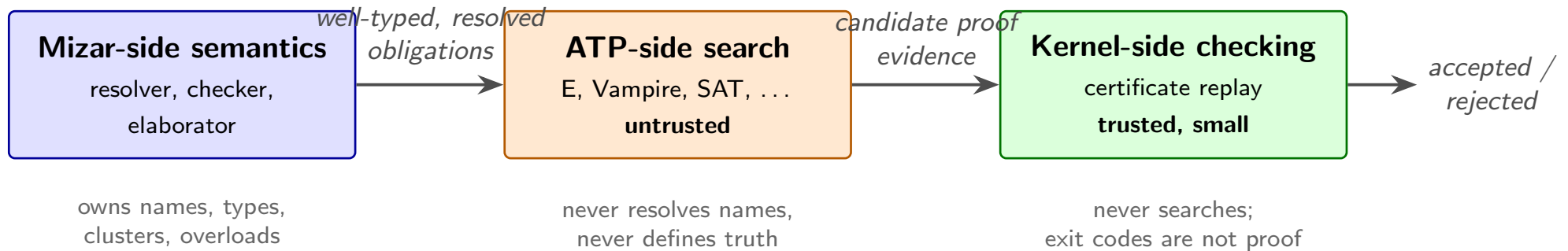
Key Points:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.
- MizAR and MPTP research already showed that ATP search is powerful on MML premises; the open question is trust, not power.

Key Phrase:

*How do we get modern proof search
without trusting the searcher?*

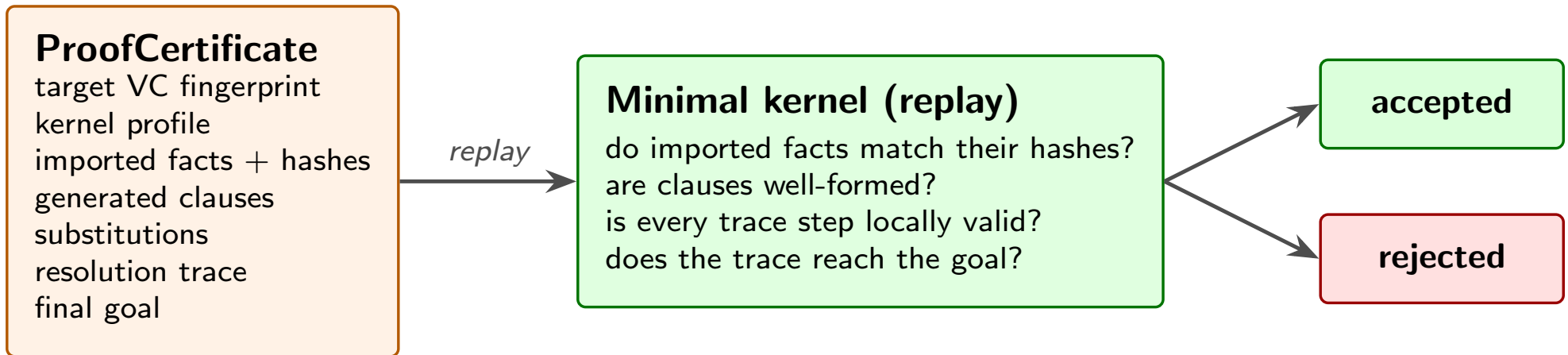
5.2 - The Evo Answer: A Reasoning Boundary



Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel never searches for proofs;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

5.3 - Certificates, Not Success Bits (1/2)



*solver exit codes never decide acceptance —
hashes link the proof to its dependency slice*

5.3 - Certificates, Not Success Bits (2/2)

Key Points:

- accepted evidence is normalized into replayable certificate data;
- the kernel checks imported facts, substitutions, clause well-formedness, and the resolution/SAT trace. It does not trust a solver's exit code. Unsoundness in search cannot cause unsoundness in accepted results;
- hashes link an accepted proof to its dependency slice. The proof cannot be reused with another slice without checking.

5.4 - The Same Boundary Controls AI

Legacy Mizar **exact MML excerpt**

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

Message:

- Citation repair is a standard safe AI edit. It proposes a missing or more precise by reference. The edit is local to the source and keeps the meaning. The verifier checks it just like a human edit.

5.5 - Edit Classes: Green, Yellow, Red

deep dive

Class	Examples	Policy
Green	add citation, insert qua, info annotation	can be proposed automatically; still verified
Yellow	add import, local lemma, registration	proposed with human review
Red	weaken theorem, change definition, add axiom	forbidden to ordinary agents

Forbidden repair **sketch**

```
theorem
  for x be Nat holds x + 0 = x or x = 0;
```

Message:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

5.6 - What Is Preserved, What We Ask

Preserved:

- the de Bruijn discipline: a small checker judges everything;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Bialystok:

- Is the SAT/resolution certificate approach convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

6.1 - The Problem

Key Points:

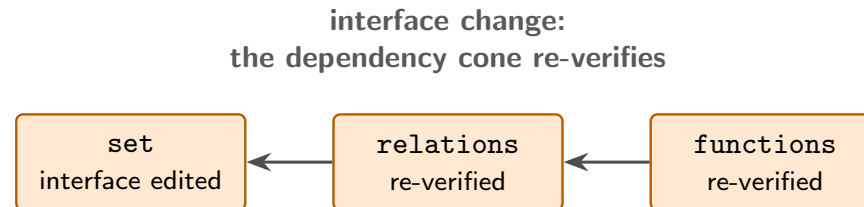
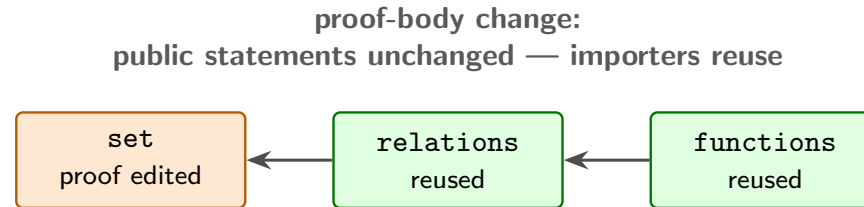
- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- These costs follow from using the article as the unit in a large library. They are not errors in the current design.

6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

Key Points:

- hash the source, the dependency interfaces, the generated obligations, and the proof witnesses;
- reuse a result only when fingerprints and verifier policy match;
- a proof-body-only change does not rebuild importers, because public statements and statuses are unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)



reuse requires matching fingerprints: source, dependency interfaces, obligations, proof witnesses — and a clean build must reproduce every acceptance

*Cache reuse is never proof authority.
A clean build must reproduce every acceptance.*

6.3 - The Evo Answer: A Memory Contract

deep dive

resident memory should scale with:

- active source
- imported public interfaces
- import-filtered indexes
- active module obligations

not with:

- imported proof bodies
- private lemmas outside the interface
- registration data outside the import closure

Message:

- Interfaces are loaded; proof bodies are not. This is what makes whole-MML editing sessions possible on ordinary hardware.

6.4 - What Is Preserved, What We Ask

Preserved:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Bialystok:

- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

7.1 - The Problem, Part One: Schemes Use Separate Rules

Legacy Mizar **exact MML excerpt**

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Key Points:

- schemes support second-order patterns (induction, separation, replacement). They work, but they use a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

7.2 - The Problem, Part Two: Copy-Paste Algebra

Key Points:

- `addMagma` and `multMagma` are the same mathematics twice, related only by naming convention (we saw this in story 2);
- polynomial rings, vector spaces, and matrix theories are written again for each carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for `+` and `*` separately.

Message:

- Without a generics mechanism, the library needs duplicate articles. Each copy needs maintenance.

7.3 - The Evo Answer: Templates

Mizar Evo **specification example**

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

Key Points:

- a template is an ordinary definition block whose leading `let` binds parameters: types, values, predicates, or functors;
- one mechanism covers structures, modes, functors, predicates, theorems, registrations, and algorithms;
- readable short forms remain: `Module over R` is an automatic synonym for `Module[R]`, `Subset of X` for `Subset[X]`.

Mizar Evo **specification example**

```
definition
  let T be type extends commutative Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Message:

- type extends commutative Magma states what the parameter must provide before any proof search begins;
- the theorem is proved once, for every commutative operation.

Instantiation **specification example**

```
PermProduct[AddMagma]           :: additive form
PermProduct[commutative MulMagma] :: multiplicative form

let R be commutative Ring;
PermProduct[R qua AddMagma]      :: R's additive view
PermProduct[R qua MulMagma]     :: R's multiplicative view
```

Message:

- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic * displays as + under the additive instantiation.

Mizar Evo **specification example**

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- instantiation is explicit bracket syntax, so tools and artifacts see exactly which instance a proof uses.

7.7 - What Is Preserved, What We Ask

Preserved:

- scheme-style reasoning keeps the same power;
- of / over phrasing keeps mathematical prose readable;
- first-order discipline: templates are checked instantiation, not a new logic.

Questions for Bialystok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with of/over as display forms?
- The specification limits inference. A parameter is inferred only when the declared argument types determine it uniquely. qua views are never inferred. Does this rule suit real MML idioms, or does it require too many explicit [T]?

8.1 - The Problem

Key Points:

- Current Mizar can define Gcd and prove its theory - but cannot compute `gcd(48, 18)`; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still meets our needs.

Key Phrase:

*The library describes computation.
It cannot perform or export it.*

8.2 - The Evo Answer: Algorithms With Contracts

Mizar Evo **specification example** (condensed from spec section 20.12)

```
definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
    requires a >= 1 & b >= 1
    ensures result = Gcd(a, b)
  do
    var x := a; var y := b;
    while y <> 0 do
      invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
      decreasing y;
      const r := x mod y; x := y; y := r;
    end;
    return x;
  end;
end;
```

- ensures and the invariant cite the mathematical Gcd: no circularity.

8.3 - The Evo Answer: Proof By Computation

Mizar Evo **specification example**

```
theorem EuclidGcd12_8:  euclid_gcd(12, 8)  = 4  by computation;  
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;  
  
theorem Fact10: factorial(10) = 3628800  
proof  
  thus thesis by computation(steps: 100000);  
end;
```

Key Points:

- the Mizar Virtual Machine (MVM) evaluates ground goals during verification, under explicit step, time, and depth budgets;
- only ground equalities and ground predicates qualify; everything else still requires a classical proof;
- imported algorithms are opaque: downstream proofs may use only their ensures contract, never their body.

Mizar Evo **specification example**

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
    decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end;
end;
```

Key Points:

- ordinary func definitions are definitional extensions: never recursive;
- once its termination obligations are proved, a terminating algorithm is promoted to a functor, usable in any proof;
- this is the only way to add recursion to the mathematical layer. It requires a proof.

Key Points:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation is MVM replay under budgets, not solver trust;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.

Message:

- Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

8.6 - What Is Preserved, What We Ask

Preserved:

- the theorem language is unchanged; algorithms live in definition blocks and interact with proofs only through contracts and verified promotion;
- first-order set-theoretic foundations stay intact.

Questions for Bialystok:

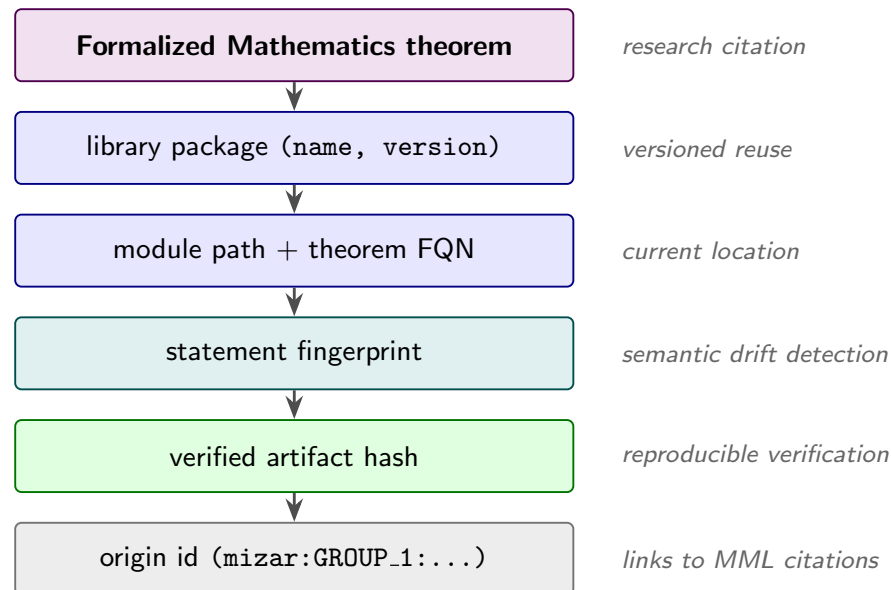
- Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by computation would immediately shorten real proofs?
- Which extraction targets matter first, if any?

9.1 - The Problem

Key Points:

- Formalized Mathematics gives Mizar a rare feature: a research journal linked to a formal library, with articles that people can cite.
- But article identity is closely tied to library organization. Refactoring the library can conflict with the structure of published articles. Package reuse has no identity for journal citations.
- A written explanation needs an order that helps readers. Reuse needs dependency order. One structure cannot serve both as well as possible.

9.2 - The Evo Answer: Linked, Not Merged



Message:

- each layer answers a different question: research citation, current location, semantic drift detection, reproducible verification, and links to MML and past Formalized Mathematics volumes.

9.3 - Who Gains What

deep dive

Audience	Gain
readers	written explanations come first; formal source is one click away
maintainers	refactoring no longer rewrites published explanations
authors	articles cite stable identities, not file layouts
AI tools	prose for retrieval, fingerprints for exact context

9.4 - What Is Preserved, What We Ask

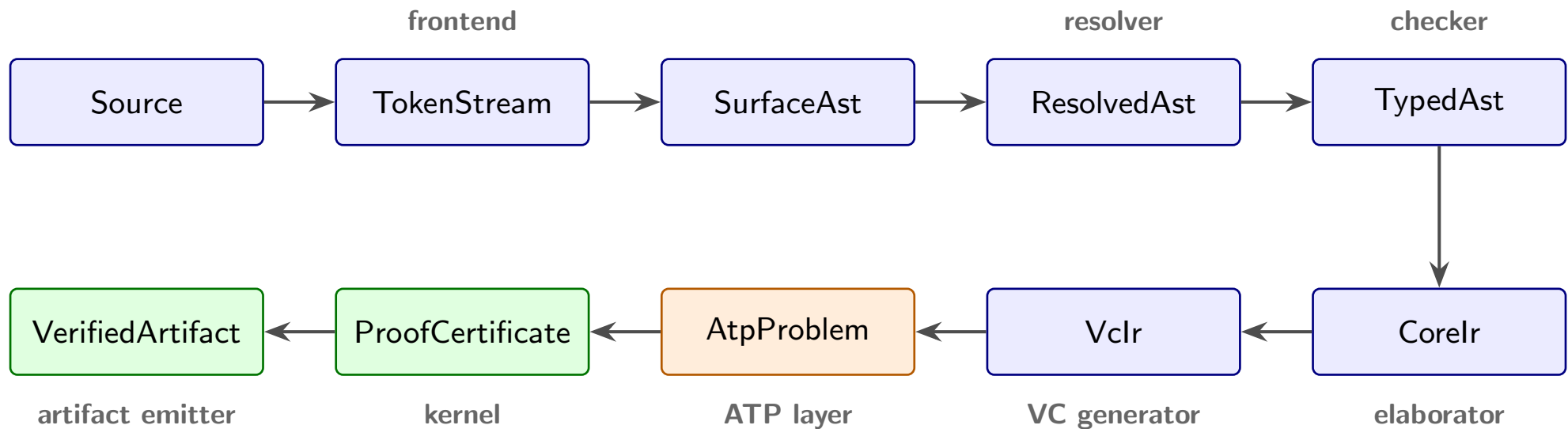
Preserved:

- Formalized Mathematics remains a real journal with review and written explanations;
- origin metadata keeps the link to every existing MML citation.

Questions for Bialystok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

10.1 - The Pipeline



Message:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes. That is the main purpose.

10.2 - Responsibility Split

deep dive

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

10.3 - Pipeline Stages For The Eight Stories

Story	Pipeline stage
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, certificates
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Message:

- The eight stories describe one pipeline. Each starts with a different problem that users face.

Key Phrase:

*Reject what must not pass
before
Accept everything that should pass*

Key Points:

- soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
- with these tests in place, we add coverage for accepted language forms.

11.1 - Where The Project Stands Today

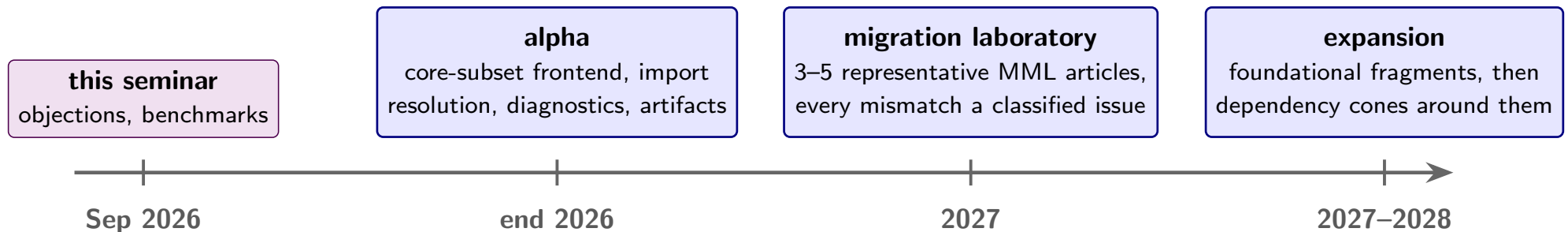
Key Points:

- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (doc/spec/);
- architecture specification: 24 documents covering the pipeline, kernel, certificate format, and AI agent interface (doc/design/architecture/);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is split into small tasks that can be verified separately.

Message:

- The planned end-of-2026 alpha is a milestone in work already under way.

11.2 - Migration Needs Research



- ① End of 2026, alpha: core-subset frontend and parser, import and module resolution prototype, structured diagnostics, early artifacts.
- ② 2027, migration laboratory: translate 3-5 representative MML articles by hand and by script. Classify and record every mismatch.
- ③ 2027-2028, expansion: foundational set and relation fragments; then algebraic structures and dependency cones around successful fragments.

Non-goals for the alpha:

- full MML verification, final compatibility layer, stable AI protocol.

Key Points:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP-plus-certificate;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

Policy:

- preserve theorem identity through origin metadata;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

Risk	How to reduce it
compatibility work takes all our time	small representative parts first; no all-at-once translation
registrations behave differently	trace artifacts and comparison reports early
package layout breaks journal links	origin metadata, article-to-library identifiers
AI edits hide migration mistakes	Red edits stay forbidden; verifier artifacts required

11.5 - What September 2026 Should Produce

Key Points:

- a list of migration benchmark articles in priority order;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

11.6 - What We Ask Of You

Questions:

- Which MML articles are small but structurally representative?
- Which idioms matter to the Mizar community, beyond their technical use?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is a serious project?

12.1 - The Main Question, Again

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

12.2 - Closing

Final slide:

*Update Mizar Evo where a larger library needs it.
Keep the current design where it defines Mizar's mathematical identity.*

Backup A. Prepared Exact Examples (1/2)

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Structure definition	algstr_0.miz	37-40	Frames 0.2, 3.1
Article environment	algstr_0.miz	15-25	Frame 2.1
Registration/cluster	algstr_0.miz	104-109	Frame 4.1

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Proof citation	struct_0.miz	637-643	Frame 5.4
Induction scheme	nat_1.miz	near 90	Frame 7.1

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Backup A. Prepared Exact Examples (2/2)

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Backup B. Specification Reference Map (1/2)

The slides show examples only. The specification is the authority for grammar and semantics. This map replaces the EBNF shown in earlier drafts.

Topic	Specification source (under doc/spec/en/)
Modules and imports	12.modules_and_namespaces.md
Structures and inheritance	05.structures.md
Attributes and modes	06.attributes.md, 07.modes.md
Registrations and reductions	17.clusters_and_registrations.md
Templates and schemes	18.templates.md
Algorithms and MVM	20.algorithm_and_verification.md
Packages and artifacts	23.package_management_and_build_system.md

Backup B. Specification Reference Map (2/2)

Topic	Specification source (under doc/spec/en/)
Cross-chapter grammar	<code>appendix_a.grammar_summary.md</code>
Worked library sketches	<code>sample_codes.md</code>

Backup C. Diagram List (1/2)

Required diagrams for the final deck (figures/*.tex, TikZ standalone;
build each with pdflatex inside figures/):

- ① Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- ② Environment-to-import migration (story 1). [done: figures/envIRON_migration.pdf, used in frame 2.5]
- ③ Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- ④ Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]
- ⑤ Certificate object and replay (story 4). [done: figures/certificate_replay.pdf, used in frame 5.3]

Backup C. Diagram List (2/2)

build each with pdflatex inside figures/):

- ① Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- ② Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- ③ Full pipeline with story overlay (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- ④ Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

Backup D. Possible Paper Outline (1/2)

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- 1 Introduction: why Mizar needs evolution now.
- 2 Mizar as baseline: readability, MML, Formalized Mathematics.
- 3 Design principles and the three goals.
- 4 Language evolution: dependencies, structures, registrations, templates.
- 5 Verifier architecture, certificates, and the small kernel.
- 6 Verified computation and the MVM.
- 7 AI-safe proof development.

Backup D. Possible Paper Outline (2/2)

Possible sections:

- ① Package-based library and publication workflow.
- ② Migration plan and evaluation measures.
- ③ Related work and plans for working together.

Backup E. Reviewer Checklist

Use this checklist before converting to Beamer:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism explain why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red AI edits clearly forbidden?
- Are migration claims measurable?

Use this checklist before converting to Beamer:

- Is EBNF absent from all frames?

Backup F. Possible Objections (1/2)

Prepared answers to other possible objections:

Objection	Prepared answer
Why not improve current Mizar incrementally?	The problems concern boundaries: article granularity, monolithic trust, and implicit environments. These boundaries cannot change in small steps. The rest of the design stays close to Mizar.
What happens to authorship and credit of MML articles?	Origin metadata keeps article identity and authorship during migration. The pub namespace keeps published articles unchanged so they can still be cited.
Is this a fork of the community?	It is a proposal to the community; this visit is its first review. The namespace governance model assumes that the Mizar team controls the <code>mm1</code> root.

Backup F. Possible Objections (2/2)

Prepared answers to other possible objections:

Objection	Prepared answer
What about GPL / CC-BY-SA obligations?	Migration preserves license and attribution of MML content; toolchain licensing is open for discussion.
Are the claims about AI too strong?	AI assistance is an optional layer that never enters the trusted base; every proposal also works without it.
Why a new kernel instead of the existing checker?	Certificate replay needs a small, auditable core. The existing checker's semantics remain the reference for obligation generation.